the words that come before it. Hence, it represents the complete sentence.

```
value1 = tf.transpose(value, [1, 0, 2])
lstm_output  =  tf.gather(value1, int(value1.get_shape()[0])
- 1)
```

Just as we obtained a fixed-length representation for Question 1, we also created a fixed-length representation for Question 2 using a second LSTM. The last stage output from each of the two LSTMs (one LSTM for each of the two questions) represents the input question representation. We can then concatenate these two representations and feed it to the multilayer perceptron classifier.

An MLP classifier is nothing but a fully connected multilayer feed forward neural network. Given that we have a two-class prediction problem, the last stage of the MLP classifier is a two-unit softmax, whose output gives the probabilities for each of the two output classes. Here is the skeletal code for an MLP classifier with three densely connected feed forward layers, with 256, 128 and two units each:

```
predict_layer_one_out = tf.layers.dense(lstm_output,

units=256,

activation=tf.nn.relu,

name="prediction_layer_one")
predict_layer_two_out = tf.layers.dense(dropout_predict_
layer_one_out,

units=128,

activation=tf.nn.relu,

name="prediction_layer_two")
predict_layer_logits = tf.layers.dense(predict_layer_two_out,

units=2,  name="final_output_layer")
```

Here are a couple of questions for our readers to think about:

- How do you decide on the number of hidden layers and the number of units in each hidden layer for your MLP classifier?
- In our problem, we are doing binary classification as we need to predict whether a question is duplicate or not. How would this code change if you had to predict one out of K different classes (for example, if you are trying to predict which category a particular question may belong to)?

The last layer output is used as input to a TensorFlow network loss computation node, which computes the cross-entropy loss using the ground truth labels as shown below:

```
loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(
                                                logits=predict_
layer_logits, labels=labels))
optimizer = tf.train.AdamOptimizer().minimize(loss)
```

The network is then trained with ground truth labels during the training phase to select network weights such that cross-entropy loss is minimised.

We also need to add code that can compute the accuracy during the training phase.

As we had discussed in earlier columns on neural networks, gradient descent techniques are typically used to learn the network parameters/weights. Typically, batch gradient descent is used for weight updates during the training process. Here are a couple of questions for our readers:

- Why do we prefer batch gradient descent over full gradient descent or stochastic gradient descent?
- How do you choose a good batch size for your implementation?

Once we have built the neural network model, we can train our model with labelled examples. Remember that each training loop consists of going over all the training samples once. This is typically known as an 'epoch'. Each epoch consists of several batch-sized runs, wherein at the end of each batch, the gradients computed are used to update the network weights/parameters. In order to ensure that the network is learning correctly, we need to measure the total loss at the end of each epoch and verify that the total loss is decreasing at the end of each successive epoch.

Now we need to decide when we should stop the training process. One simple but naïve way of stopping the training process is after a fixed number of epochs. Another option is to stop training after we have reached a training accuracy of 100 per cent.

Here is a question for our readers: While we can decide to stop training after some fixed number of epochs or after a training accuracy of 100 per cent, what are the disadvantages associated with each of these approaches? We will discuss more on this topic as well as on the inference phase in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at *sandyasm_AT_yahoo_DOT_com*. Wishing all our readers happy coding until next month! END

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist at Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training (India)' at *http://www.linkedin.com/groups?home=&gid=2339182.*